

Звіт
з дисципліни «Проектування кібер-фізичних систем»
на тему «Побудова генетичного алгоритму »

студента групи ТВ-33
Домарацького Дмитра

Завдання:

Запрограмувати вирішення обраної задачі за допомогою генетичного алгоритму. Дати можливість обирати кількість та якість початкової популяції чи декількох популяцій. Кількість ітерацій має теж задаватись, включно з опцією “до останнього живого”. Виводити статистику популяцій після кожної ітерації.

Розв'язання:

У мене п'ятий варіант (непарний), оберу першу пропозицію: звичайний генетичний алгоритм (Simple Genetic Algorithm): Заснований на базових операціях кросоверу, мутації та вибору. Реалізація буде у VisualStudio на мові програмування Python. Будемо вирішувати задачу комівояжера (Travelling Salesman Problem): мінімізація відстані, яку потрібно подолати продавцеві, щоб відвідати кожного клієнта точно один раз і повернутися в точку виходу. Згідно з завданням, у алгоритмі повинні бути наступні кроки:

- 1) Ініціалізація популяції: Створення початкової популяції із випадковими особинами (кандидатами на рішення).
- 2) Оцінка придатності (fitness evaluation): Обчислення значень придатності для кожної особини у популяції на основі поставленої задачі.
- 3) Вибір батьків: Вибір особин для формування нащадків (на основі їхньої придатності). Можливі різні методи вибору, такі як випадковий вибір, вибір турнірний, вибір за рулеткою тощо.
- 4) Створення нащадків: Застосування генетичних операторів (кросоверу та мутації) до обраних батьків для створення нової популяції нащадків.
- 5) Оцінка придатності нової популяції: Обчислення значень придатності для кожної особини у новій популяції.
- 6) Вибір для наступної популяції: Вибір особин для наступної популяції, як правило, здійснюється за принципом елітарності (збереження найкращих особин) та може включати також нових нащадків.
- 7) Умова зупинки: Перевірка умови зупинки, такої як досягнення заданої кількості ітерацій або зміна величини придатності менше за певний поріг.

Повторення: Повторення кроків 3-7, доки не буде виконана умова зупинки.

- 8) Вивід результатів: Повернення найкращого знайденого рішення або найкращої особини у популяції.

Для цього завдання використаємо бібліотеку DEAP, вона зручна і уже містить багато вбудованих функцій для роботи з генетичними алгоритмами, наприклад, для кросоверу та мутації, а також є функції з запуском самих генетичних алгоритмів та збором з них статистичних даних, які потім можна зібрати і вивести візуально та динамічно за допомогою бібліотеки matplotlib (рисунок 1, 2).

```
from deap import base, creator, tools, algorithms
import random, numpy as np, matplotlib.pyplot as plt
from matplotlib.animation import ArtistAnimation
import math

# ----- Налаштування -----
NUM_CITIES = 20
POPULATION_SIZE = 100
P_CROSSOVER = 0.9
P_MUTATION = 0.2
MAX_GENERATIONS = 100

# ----- Генерація міст -----
cities = [(random.randint(0, 100), random.randint(0, 100)) for _ in range(NUM_CITIES)]

def euclidean(p1, p2):
    return math.hypot(p1[0] - p2[0], p1[1] - p2[1])

def total_distance(individual):
    dist = 0
    for i in range(len(individual)):
        city1 = cities[individual[i]]
        city2 = cities[individual[(i + 1) % len(individual)]]
        dist += euclidean(city1, city2)
    return (dist,)

# ----- DEAP -----
creator.create("FitnessMin", base.Fitness, weights=(-1.0,))
creator.create("Individual", list, fitness=creator.FitnessMin)
```

Рисунок 1 — код програми

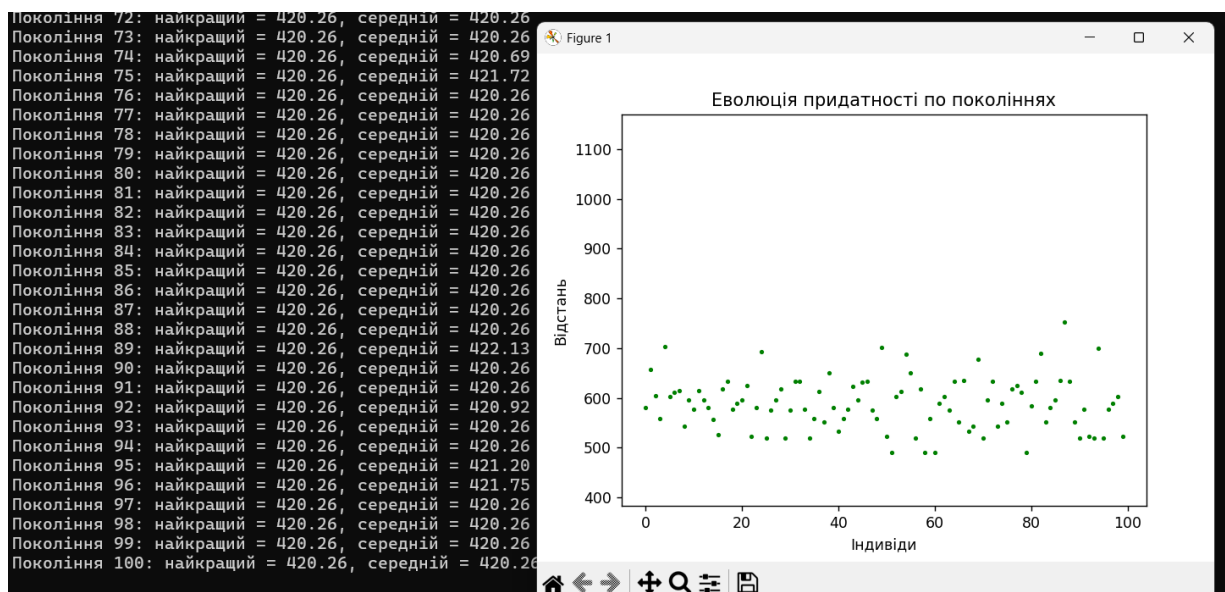


Рисунок 2 — результат виконання

Необхідно вміти пояснити, що таке генетичний алгоритм, який клас задач він вирішує та чому такі задачі відносяться до кіберфізичних систем.

Генетичний алгоритм — це еволюційний алгоритм пошуку, що використовується для вирішення задач оптимізації і моделювання шляхом послідовного підбору, комбінування і варіації шуканих параметрів з використанням механізмів, що нагадують біологічну еволюцію.

Генетичні алгоритми застосовується для вирішення наступних задач:

1. Оптимізація функцій
2. Оптимізація запитів в базах даних
3. Різноманітні задачі на графах (задача комівояжера, розфарбування)
4. Налаштування і навчання штучної нейронної мережі
5. Задачі компоновки
6. Створення розкладів
7. Ігрові стратегії
8. Апроксимація функцій
9. Штучне життя
10. Біоінформатика (згортання білків)
11. Синтез конструкцій антен

Генетичні алгоритми та задачі, які вони вирішують, відносяться до кіберфізичних систем, оскільки, по-перше, вони реалізують кібернетичну частину таких систем — обчислювальний компонент, сам алгоритм, що відповідає за збір, обробку та аналіз даних, а також за прийняття оптимальних рішень, по-друге, задачі, які вирішуються за допомогою генетичних алгоритмів, безпосередньо пов'язані з фізичною частиною — тобто з реальними об'єктами, процесами чи середовищем, якими потрібно керувати, це можуть бути: маршрути руху роботів, розклади виробництва, розміщення обладнання або інші аспекти, які мають фізичне втілення та обмеження.

https://uk.wikipedia.org/wiki/%D0%93%D0%B5%D0%BD%D0%B5%D1%82%D0%B8%D1%87%D0%BD%D0%B8%D0%B9_%D0%B0%D0%BB%D0%B3%D0%BE%D1%80%D0%B8%D1%82%D0%BC